

Spring MVC

What is JSTL ?

- JSTL is the JavaServer Pages Standard Tag Library.
- It provides ready-made tags to replace Java code in JSP.
- It helps perform tasks like loops, conditions, and formatting.
- It makes JSP pages cleaner, simpler, and easier to maintain.

Why JSTL is used ?

- JSTL removes Java scriptlets and makes JSP clean and readable.
- It provides easy tags for loops, conditions, and iteration.
- It supports formatting, internationalization, and reduces code duplication.
- It helps maintain MVC by keeping business logic out of JSP.

JSTL Tag Libraries :-

Library Name	Prefix	Purpose / Use
Core	c	Variables, loops, conditions
Formatting	fmt	Numbers, dates, currency, internationalization
SQL	sql	Simple database operations
XML	x	XML parsing and processing
Functions	fn	String functions (length, contains, etc.)

Core JSTL Tags (c tag) :-

URI:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
```

Core tags include:

A) Variable tags -

```
<c:set var="x" value="10"/>    // Set a variable value
```

```
<c:remove var="x"/>           // Remove a variable
```

B) Conditional tags -

```
<c:if test="${age > 18}">Adult</c:if>    // Conditional check (if statement)

<c:choose>    // Multiple conditions (switch-like)

    <c:when test="${marks >= 35}">Pass</c:when>    // Condition inside choose

    <c:otherwise>Fail</c:otherwise>    // Default block inside choose

</c:choose>
```

C) Loop tags -

```
<c:forEach var="student" items="${list}">    // Loop/iterate over a collection

    ${student.name}

</c:forEach>
```

How JSTL Helps in MVC :-

- Keeps controller logic in Java classes, not in JSP.
- Maintains presentation logic in JSP using JSTL tags.
- Keeps business logic in the service layer.
- Ensures JSP contains no Java code—only clean JSTL tags.

How to Enable JSTL :-

1. Add the JSTL Maven dependency.

```
<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>jstl</artifactId>
  <version>1.2</version>
</dependency>
```

2. Include the JSTL taglib directive at the top of the JSP page.

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
```

Benefits of Using JSTL :-

- Makes JSP cleaner and removes scriptlets.
- Easy to read, maintain, and debug.
- Works well with MVC frameworks like Spring MVC.
- Reduces errors and speeds up development.
- Supports formatting and internationalization.

Implementation of Spring MVC Application :-

index.jsp –

```
<html>
<body>

    <!-- <form action="add" method="post">
        <input type="text" name="num1" placeholder="Enter number1"><br><br>
        <input type="text" name="num2" placeholder="Enter number2"><br><br>

        <button type="submit">Submit</button>
    </form> -->

    <form action="save" method="post">
        <input type="text" name="fullname" placeholder="Enter fullname"><br><br>
        <input type="email" name="email" placeholder="Enter email"><br><br>
        <input type="password" name="password" placeholder="Enter password"><br><br>
        <textarea name="address" id="address" placeholder="Enter address"></textarea><br><br>

        <button type="submit">Submit</button>
    </form>

</body>
</html>
```

pom.xml –

```
https://maven.apache.org/xsd/maven-4.0.0.xsd (xsi:schemaLocation)
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.demo</groupId>
  <artifactId>Maven_Sample_Project2</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>war</packaging>

  <dependencies>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-webmvc</artifactId>
      <version>5.3.30</version>
    </dependency>

    <dependency>
      <groupId>javax.servlet</groupId>
      <artifactId>javax.servlet-api</artifactId>
      <version>4.0.1</version>
      <scope>provided</scope>
    </dependency>

    <dependency>
      <groupId>org.postgresql</groupId>
      <artifactId>postgresql</artifactId>
      <version>42.6.0</version>
    </dependency>

    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-jdbc</artifactId>
      <version>5.3.32</version>
    </dependency>

    <dependency>
      <groupId>javax.servlet</groupId>
      <artifactId>jstl</artifactId>
      <version>1.2</version>
    </dependency>

  </dependencies>

  <build>
    <finalName>Maven_Sample_Project2</finalName>
  </build>
</project>
```

web.xml –

Bind to grammar/schema...

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee" version="3.1">

    <servlet>
        <servlet-name>spring</servlet-name>
        <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
        <load-on-startup>1</load-on-startup>
    </servlet>

    <servlet-mapping>
        <servlet-name>spring</servlet-name>
        <url-pattern>/</url-pattern>
    </servlet-mapping>

</web-app>
```

Student.java –

```
package com.demo.models;

public class Student {
    private int id;
    private String fullname;
    private String email;
    private String password;
    private String address;

    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getFullname() {
        return fullname;
    }
    public void setFullname(String fullname) {
        this.fullname = fullname;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
    public String getPassword() {
        return password;
    }
    public void setPassword(String password) {
        this.password = password;
    }
    public String getAddress() {
        return address;
    }
    public void setAddress(String address) {
        this.address = address;
    }
}
```

spring-servlet.xml –

```
http://www.springframework.org/schema/beans/spring-beans.xsd (xsi:schemaLocation) | http://www.
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:mvc="http://www.springframework.org/schema/mvc"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
         http://www.springframework.org/schema/beans
         http://www.springframework.org/schema/beans/spring-beans.xsd
         http://www.springframework.org/schema/context
         http://www.springframework.org/schema/context/spring-context.xsd
         http://www.springframework.org/schema/mvc
         http://www.springframework.org/schema/mvc/spring-mvc.xsd">

    <context:component-scan base-package="com.demo.controllers"></context:component-scan>

    <!-- View Resolver -->
    <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
        <property name="prefix" value="/WEB-INF/pages/" />
        <property name="suffix" value=".jsp" />
    </bean>

    <bean id="dataSource" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
        <property name="driverClassName" value="org.postgresql.Driver" />
        <property name="url" value="jdbc:postgresql://localhost:5433/springmaven1" />
        <property name="username" value="postgres" />
        <property name="password" value="4518" />
    </bean>

    <bean id="jdbcTemplate" class="org.springframework.jdbc.core.JdbcTemplate">
        <property name="dataSource" ref="dataSource" />
    </bean>

    <bean id="dao" class="com.demo.dao.StudentDAO">
        <property name="template" ref="jdbcTemplate"></property>
    </bean>

    <!-- Enable Spring MVC -->
    <mvc:annotation-driven />

</beans>
```

StudentController.java –

```
package com.demo.controllers;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;

import com.demo.dao.StudentDAO;
import com.demo.models.Student;

@Controller
public class StudentController {

    // @PostMapping("/add")
    // public String addition(@RequestParam int num1, @RequestParam int num2, Model m)
    // {
    //     int result=num1+num2;
    //     System.out.println("Addition= "+result);
    //     m.addAttribute("result",result);
    //     return "welcome";    //welcome.jsp
    // }

    // @GetMapping("/welcome")
    // public String wel() {
    //     return "welcome";    //welcome.jsp
    // }

    @Autowired
    StudentDAO sdao;

    @PostMapping("/save")
    public String save(@ModelAttribute Student s) {
        boolean result=sdao.registration(s);
        if(result) {
            System.out.println("Registration Successfully Completed");
            return "redirect:/studentsdata";
        }else {
            System.out.println("Fail");
            return "redirect:/";
        }
    }

    @GetMapping("/studentsdata")
    public String studentsdata(Model m) {
        List<Student> list=sdao.getallstudents();
        m.addAttribute("list",list);
        return "studentsdata";    //studentsdata.jsp
    }

    @GetMapping("/deletestudent/{id}")
    public String deletestudent(@PathVariable int id) {
        sdao.delete(id);
        return "redirect:/studentsdata";
    }
}
```

Studentsdata.jsp –

```
package com.demo.dao;

import java.util.List;

import org.springframework.jdbc.core.JdbcTemplate;

import com.demo.models.Student;

public class StudentDAO {

    JdbcTemplate template;

    public void setTemplate(JdbcTemplate template) {
        this.template = template;
    }

    public boolean registration(Student s) {
//        String sql="insert into student(full_name,email,password)
//        values('"+s.getFullname()+"', '"+s.getEmail()+"', '"+s.getPassword()+"', '"+s.getAddress()+"')";
//        template.update(sql);

        String sql="insert into student(full_name,email,password,address) values(?,?,?,?)";
        template.update(sql,s.getFullname(),s.getEmail(),s.getPassword(),s.getAddress());
        return true;
    }

    public List<Student> getallstudents(){
        String sql = "select * from student order by id DESC";

        return template.query(sql,(rs,numberofrows)->{
            Student s=new Student();
            s.setId(rs.getInt("id"));
            s.setFullname(rs.getString("full_name"));
            s.setEmail(rs.getString("email"));
            s.setPassword(rs.getString("password"));
            s.setAddress(rs.getString("address"));
            return s;
        });
    }

    public void delete(int id) {
        String sql="delete from student where id=?";
        template.update(sql,id);
    }
}
```

StudentDAO.java –

```
package com.demo.dao;

import java.util.List;

import org.springframework.jdbc.core.JdbcTemplate;

import com.demo.models.Student;

public class StudentDAO {

    JdbcTemplate template;

    public void setTemplate(JdbcTemplate template) {
        this.template = template;
    }

    public boolean registration(Student s) {
//        String sql="insert into student(full_name,email,password)
//        values('"+s.getFullname()+"','"+s.getEmail()+"','"+s.getPassword()+"','"+s.getAddress()+"')";
//        template.update(sql);

        String sql="insert into student(full_name,email,password,address) values(?,?,?,?)";
        template.update(sql,s.getFullname(),s.getEmail(),s.getPassword(),s.getAddress());
        return true;
    }

    public List<Student> getallstudents(){
        String sql = "select * from student order by id DESC";

        return template.query(sql,(rs,numberofrows)->{
            Student s=new Student();
            s.setId(rs.getInt("id"));
            s.setFullname(rs.getString("full_name"));
            s.setEmail(rs.getString("email"));
            s.setPassword(rs.getString("password"));
            s.setAddress(rs.getString("address"));
            return s;
        });
    }

    public void delete(int id) {
        String sql="delete from student where id=?";
        template.update(sql,id);
    }
}
```

Output –

localhost:8081/Maven_Sample_Project2/index.jsp

Enter fullname

Enter email

Enter password

Enter address

Submit

localhost:8081/Maven_Sample_Project2/index.jsp

Sagar Kate

sagarkate374@gmail.com

....

Koregaon

Submit

Console Based –

Registration Successfully Completed

localhost:8081/Maven_Sample_Project2/studentsdata

Students Data

ID	Fullname	Email	Password	Address	
1	Sagar Kate	sagarkate374@gmail.com	8527	Koregaon	Delete
2	Atharv Jadhav	atharvjadhav454@gmail.com	8520	Pune	Delete
3	Omkar Pawar	omkar@gmail.com	4580	Kudal,Satara	Delete

After Delete Id –

localhost:8081/Maven_Sample_Project2/studentsdata					
Students Data					
Id	Fullname	Email	Password	Address	
1	Sagar Kate	sagarkate374@gmail.com	8527	Koregaon	Delete
2	Atharv Jadhav	atharvjadhav454@gmail.com	8520	Pune	Delete

Database Based –

	id [PK] integer	full_name character varying (100)	email character varying (100)	password character varying (100)	address character varying (255)
1	5	Atharv Jadhav	atharvjadhav454@gmail.c...	8520	Pune
2	6	Sagar Kate	sagarkate374@gmail.com	8527	Koregaon